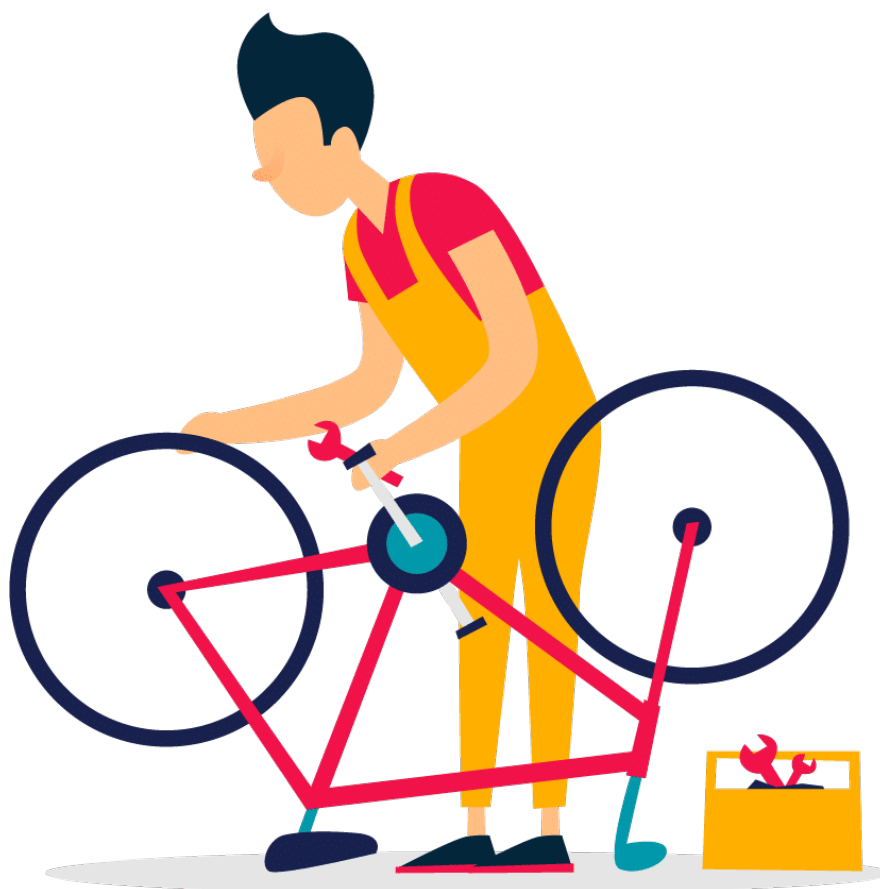




Livrables du projet

Application HOMECYCL'HOME

Préparé pour : LeCycleLyonnais
Préparé par : Yannick Biot
12/01/2026

1. CAHIER DES CHARGES FONCTIONNEL	3
2. CAHIER DES CHARGES TECHNIQUES	5
3. MACRO PLANNING	11
4. BACKLOG / RELEASE / SPRINT	16
5. INFRASTRUCTURE	20
6. MAQUETTAGE	23
7. MODÉLISATION	24
9. APPLICATION	30
10. DÉPLOIEMENT EN MODE DEVOPS	31
10. TESTS (UNITAIRES, FONCTIONNELS)	36
11. DOCUMENTATION	36
ANNEXE A	37
ANNEXE B : DICTIONNAIRE DE DONNÉES	38
ANNEXE C : DIAGRAMME MCD	43
ANNEXE D : DIAGRAMME MLD	44

1. CAHIER DES CHARGES FONCTIONNEL

1.1 Gestion des informations de l'entreprise

- Modifier les informations affichées de l'entreprise (coordonnées, description, etc.)

1.2 Gestion des utilisateurs

- S'authentifier sur l'application
- Créer un compte client
- Gérer les rôles (admin / technicien / client)
- Administrer les utilisateurs (CRUD)
- Gérer le profil client

1.3 Gestion des cycles du client

- Définir les types de cycles
- Ajouter / modifier / supprimer un cycle associé à un client

1.4 Gestion des prestations

- Créer / modifier / supprimer des prestations d'entretien
- Définir les tarifs associés

1.5 Gestion des produits additionnels

- Créer / modifier / supprimer des produits additionnels
- Associer un prix aux produits

1.6 Gestion des zones géographiques

- Créer / modifier / supprimer des zones géographiques
- Affecter des zones aux techniciens
- Vérifier l'éligibilité d'une zone lors de la réservation

1.7 Gestion de la planification

- Définir des créneaux disponible
- Gérer les modèles de planification
- Associer créneaux / zones / techniciens

1.8 Gestion des interventions

- Créer une intervention (suite à la réservation d'un créneau)
- Modifier ou annuler une intervention
- Consulter les interventions
- Ajouter des photos et commentaires
- Valider le paiement
- Clôturer une intervention

1.9 Gestion des exigences légales - conformité RGPD

- Consulter les Conditions Générales d'Utilisation

- Accepter les CGU lors de la création du compte
- Gérer le consentement aux cookies
- Consulter la politique de confidentialité
- Modifier ou supprimer son compte (droit à l'oubli)

2. CAHIER DES CHARGES TECHNIQUES

2.1 Architecture Frontend

Le projet HomeCycl'Home nécessite une interface responsive, multi-rôles et connectée à une API REST sécurisée. L'architecture doit permettre une séparation claire entre logique d'affichage et logique métier.

Les solutions étudiées ont été :

- Architecture découplée (React SPA + API REST séparée)
- Framework fullstack (Next.js)
- Angular + API séparée

L'architecture fullstack simplifie la mise en œuvre mais réduit la séparation explicite des responsabilités. Angular apporte un cadre structuré mais introduit une complexité supérieure au regard du périmètre du projet.

L'architecture découplée a été retenue pour :

- Sa séparation claire frontend / backend
- Sa meilleure lisibilité architecturale
- Sa cohérence avec une API REST NestJS
- Sa capacité à démontrer les compétences du référentiel

Choix retenu : React + Vite + TypeScript dans une architecture découplée.

2.2 Backend

Le projet nécessite une API REST structurée permettant de gérer les utilisateurs, les interventions, le planning, les zones géographiques et la sécurité d'accès. L'architecture backend doit être modulaire, maintenable et cohérente avec une approche Clean Architecture.

Les solutions étudiées ont été :

- Express.js
- NestJS
- Spring Boot

Express permet une mise en œuvre rapide mais nécessite une structuration manuelle de l'architecture. Spring Boot offre un cadre robuste mais introduit une complexité et un changement d'écosystème non justifiés pour le périmètre du projet.

NestJS apporte :

- Une architecture modulaire native
- Une intégration TypeScript complète
- Un système avancé de gestion des rôles et de la sécurité
- Une cohérence avec le frontend React

Choix retenu : NestJS.

2.3 Base de données

Le projet HomeCycl'Home repose sur un modèle fortement relationnel : utilisateurs, interventions, techniciens, zones géographiques, forfaits et produits additionnels. La cohérence des données et l'intégrité référentielle sont essentielles, notamment pour la gestion des créneaux et des interventions.

Les solutions étudiées ont été :

- PostgreSQL
- MySQL
- MongoDB

MongoDB, orientée document, est moins adaptée à un modèle relationnel complexe nécessitant des contraintes fortes. MySQL constitue une solution fiable mais offre moins de fonctionnalités avancées en matière d'intégrité et de conformité stricte au standard SQL.

PostgreSQL se distingue par :

- Sa robustesse et sa stabilité
- Sa gestion avancée des contraintes et transactions
- Son excellente compatibilité avec l'écosystème Node/NestJS
- Sa capacité à évoluer vers des besoins plus complexes

Choix retenu : PostgreSQL.

2.4 Architecture logicielle

Le projet implique une logique métier structurée : gestion des créneaux, affectation des zones, contrôle des rôles et validation des interventions. L'architecture doit permettre une séparation claire entre logique métier, infrastructure et couche de présentation.

Les solutions étudiées ont été :

- MVC classique
- Clean Architecture
- Architecture Hexagonale

Le modèle MVC reste simple mais limite l'isolation complète du domaine métier. L'architecture hexagonale offre une séparation très poussée mais introduit une complexité supérieure au regard du périmètre pédagogique.

La Clean Architecture permet :

- Une séparation claire entre domaine, application et infrastructure
- Une meilleure testabilité (notamment dans une logique TDD)
- Une évolutivité facilitée
- Une organisation cohérente avec NestJS

Choix retenu : Clean Architecture pragmatique adaptée à NestJS.

2.5 Stratégie d'accès aux données (ORM)

Le projet nécessite une couche d'accès aux données claire et maintenable, en cohérence avec PostgreSQL, NestJS et l'architecture Clean retenue. Les relations entre entités (interventions, utilisateurs, zones, forfaits) doivent être lisibles et sécurisées.

Les solutions étudiées ont été :

- Prisma
- TypeORM
- Sequelize

Sequelize offre une intégration historique avec Node.js mais un typage limité en TypeScript. TypeORM est bien intégré à NestJS mais présente une complexité accrue et une gestion du typage moins stricte.

Prisma se distingue par :

- La génération automatique de types TypeScript
- Un schéma centralisé clair et lisible
- Une bonne intégration avec PostgreSQL
- Une meilleure sécurité à la compilation

Choix retenu : Prisma.

2.6 API Mapping

Le projet prévoit la validation d'adresse lors de la réservation d'une intervention ainsi que l'affichage des zones couvertes. Une solution fiable de géocodage et une carte interactive sont nécessaires pour améliorer l'expérience utilisateur.

Les solutions étudiées ont été :

- Google Maps Platform
- Mapbox
- OpenStreetMap (avec Leaflet)

OpenStreetMap constitue une solution open-source intéressante mais offre une précision et un écosystème moins complets. Mapbox propose une alternative moderne et performante.

Google Maps se distingue par :

- La précision de son géocodage
- Son système d'autocomplétion performant
- La qualité de ses cartes interactives
- Une documentation et une stabilité éprouvées

Choix retenu : Google Maps Platform (Geocoding + Maps JavaScript API).

2.7 Librairie de planification

Le projet nécessite l’affichage des créneaux par technicien, la gestion des interventions et la visualisation sous forme de calendrier (jour / semaine / mois). La solution doit être compatible avec React et permettre une personnalisation suffisante.

Les solutions étudiées ont été :

- FullCalendar
- React Big Calendar
- DevExtreme Scheduler

DevExtreme offre des fonctionnalités avancées mais repose sur un modèle de licence payant peu adapté au périmètre pédagogique. React Big Calendar constitue une alternative légère et cohérente avec React.

FullCalendar se distingue par :

- La richesse de ses fonctionnalités
- Sa maturité et sa stabilité
- Sa bonne intégration avec React
- L’existence d’une version gratuite suffisante pour le projet

Choix retenu : FullCalendar (version gratuite).

2.8 Stratégie de sécurité applicative

Le projet implique la gestion de comptes clients, techniciens et administrateurs, ainsi que des données personnelles (adresse, photos, historique d’interventions). La sécurisation des accès à l’API et la protection des données sont donc essentielles.

Les solutions étudiées ont été :

- Authentification par JWT avec Refresh Token
- Sessions server-side (cookies + session store)
- Solution d’authentification externe (Auth0 / Firebase)

Les sessions server-side offrent une bonne sécurité mais sont moins adaptées à une architecture API REST découplée. Les solutions externes simplifient l’implémentation mais limitent la maîtrise complète du mécanisme d’authentification.

L’approche JWT avec Refresh Token permet :

- Une authentification stateless adaptée à une API REST
- Une gestion fine des rôles (RBAC)
- Une bonne intégration avec NestJS (Guards, stratégies Passport)
- Une maîtrise complète de la sécurité applicative

Cette stratégie est complétée par :

- Hashage des mots de passe (bcrypt)
- Validation des données (DTO + class-validator)
- Protection CORS et rate limiting

- Obligation HTTPS
- Paiement physique (aucune donnée bancaire stockée)

Choix retenu : JWT + Refresh Token + RBAC.

2.9 Repository & Stratégie CI/CD

Le projet nécessite une gestion rigoureuse du versioning, une intégration continue des tests et un déploiement automatisé, conformément à une démarche DevOps attendue dans les livrable.

Les solutions étudiées ont été :

- GitHub + GitHub Actions
- GitLab + GitLab CI
- Azure DevOps

GitLab et Azure DevOps offrent des solutions complètes mais introduisent une complexité supérieure au regard du périmètre pédagogique.

GitHub associé à GitHub Actions permet :

- Une gestion simple des branches (feature → pull request → main)
- L'automatisation des builds et des tests
- L'intégration native avec un projet Node/React
- Un déploiement automatisé vers l'environnement cible

Choix retenu : GitHub + GitHub Actions.

2.10 Stratégie de tests

Le projet comporte des règles métier critiques : gestion des créneaux, affectation des zones, contrôle des rôles et sécurisation des accès. La fiabilité du système repose sur une validation rigoureuse de ces logiques.

Les solutions étudiées ont été :

- Approche TDD (Test Driven Development) avec Jest
- Tests écrits après développement
- Tests principalement E2E (Cypress)

Les tests écrits après développement sécurisent l'application mais ne structurent pas la conception. Une approche majoritairement E2E valide le comportement global mais isole moins précisément les règles métier.

L'approche TDD permet :

- De concevoir la logique métier à partir des cas de test
- D'améliorer la qualité et la testabilité du code
- De sécuriser les règles de réservation et de gestion des rôles
- D'intégrer naturellement les tests dans la pipeline CI/CD

La stratégie retenue est :

- TDD appliqué aux cas d'usage métier
- Tests unitaires avec Jest
- Tests d'intégration API avec Supertest
- Exécution automatique dans GitHub Actions

Choix retenu : TDD + Jest + Supertest.

2.11 Containérisation & Déploiement

Le projet nécessite un environnement stable pour le déploiement en production ainsi qu'une intégration dans la pipeline CI/CD.

Les solutions étudiées ont été :

- Déploiement classique sans containerisation
- Docker simple
- Docker avec orchestration via Docker Compose

Docker Compose permet :

- La séparation des services (backend, PostgreSQL, Nginx)
- Une isolation réseau contrôlée
- Une reproductibilité de l'environnement de production
- Une intégration simplifiée dans GitHub Actions

Dans une logique pragmatique :

- Le développement sera réalisé en environnement local natif pour optimiser la productivité.
- La production sera conteneurisée via Docker Compose.

Choix retenu : Docker + Docker Compose pour la production uniquement.

3. MACRO PLANNING

3.1 Contexte et objectif du projet

CONTEXTE

Le projet HomeCycl'Home consiste en la conception et le développement d'une application web permettant la gestion d'interventions d'entretien et de réparation de cycles à domicile.

L'objectif métier est de digitaliser le processus de réservation, de planification et de suivi des interventions entre trois acteurs :

- Le **client**, qui réserve une intervention
- Le **technicien**, qui réalise l'intervention
- L'**administrateur**, qui supervise et configure l'activité

OBJECTIF DU MVP

Le MVP (Minimum Viable Product) vise à livrer une version fonctionnelle permettant de valider la valeur principale du produit :

Permettre à un client de réserver une intervention, à un technicien de la réaliser, puis d'enregistrer le paiement et de clôturer l'intervention.

Le MVP inclut uniquement les fonctionnalités indispensables au cycle métier complet :

1. Authentification et gestion des utilisateurs
2. Gestion des cycles et prestations
3. Gestion des zones géographiques
4. Planification des créneaux
5. Création, modification, annulation et clôture d'intervention
6. Déclaration du paiement

Certaines fonctionnalités secondaires (produits additionnels, gestion des informations de l'entreprise, RGPD complet) ont été volontairement exclues du périmètre MVP afin de maîtriser la charge et sécuriser la livraison.

DURÉE ET ORGANISATION

Le projet s'étend de **février à août**, avec :

- Une phase de cadrage et conception
- Quatre sprints de développement
- Une phase finale de tests, documentation et livraison

La planification repose sur une estimation en story points convertis en charge horaire (1 point ≈ 1,4 heure), permettant un pilotage réaliste du projet.

3.2 Diagramme de Gantt

[illegible]

PHASE 1 – INITIATION, CADRAGE & PLANIFICATION (FÉVRIER)

Cette phase a permis de poser les bases stratégiques et techniques du projet.

Elle comprend :

- L'analyse du besoin métier
- L'identification des acteurs et la formalisation des cas d'usage
- La conception du modèle de données (MCD)
- Les choix d'architecture technique
- La définition du backlog initial
- L'identification et la délimitation du périmètre MVP
- La réalisation du story mapping
- L'établissement de la planification prévisionnelle

Cette phase constitue le socle du projet en garantissant une vision claire du périmètre, des priorités et des choix techniques avant le démarrage du développement.

PHASE 2 – CONCEPTION FONCTIONNELLE & TECHNIQUE (MARS)

Cette phase marque la transition entre la planification et le développement fonctionnel.

Elle comprend :

- La finalisation de la conception fonctionnelle (affinage des cas d'usage et du backlog)
- La définition de l'architecture applicative (frontend, backend, base de données)
- La structuration des dépôts et de l'organisation du code
- L'implémentation du noyau du projet (architecture de base, routing, structure modulaire)
- La configuration de l'environnement de développement

- La mise en place des outils DevOps :
 - gestion de versions (Git)
 - pipeline CI (lint, build, tests)
 - configuration des variables d'environnement
 - dockerisation de l'environnement
- La préparation de l'environnement de déploiement

Cette phase vise à sécuriser le cadre technique afin de permettre un développement itératif maîtrisé lors des sprints suivants.

PHASE 3 - DÉVELOPPEMENT ITÉRATIF (AVRIL À JUILLET)

Cette phase correspond au développement progressif du MVP selon une approche Agile organisée en sprints.

Le travail est structuré en quatre sprints successifs, chacun livrant un incrément fonctionnel :

- **Sprint 1** : Mise en place de l'authentification et gestion des cycles
- **Sprint 2** : Gestion des prestations, zones géographiques et planification
- **Sprint 3** : Création et gestion des interventions
- **Sprint 4** : Finalisation du cycle métier (annulation, paiement, clôture)

Chaque sprint comprend :

- Sélection des user stories prioritaires
- Développement incrémental
- Tests unitaires et fonctionnels
- Validation de l'incrément

L'objectif de cette phase est de livrer progressivement un produit opérationnel tout en maîtrisant les risques grâce à la priorisation du MVP.

PHASE 4 – STABILISATION, TESTS GLOBAUX & DOCUMENTATION (AOÛT)

Cette phase vise à consolider le MVP avant livraison.

Elle comprend :

- Les tests fonctionnels complets du cycle métier
- La correction des anomalies identifiées
- La vérification de la cohérence des données
- La documentation technique
- La finalisation de la documentation projet

Cette phase garantit la qualité globale du produit avant remise.

PHASE 5 – LIVRAISON & SOUTENANCE

La dernière phase comprend :

- La préparation de la démonstration
- La validation finale du MVP
- La remise des livrables
- La soutenance du projet

L'objectif est de présenter une application fonctionnelle démontrant un cycle métier complet et une démarche projet structurée.

3.3 Jalons clés du projet

J1 – VALIDATION DU CADRAGE (FIN FÉVRIER)

- Use cases validés
- MCD conçu
- Architecture choisie
- MVP défini
- Planification établie

Le projet est cadré et prêt à entrer en phase technique.

J2 – NOYAU TECHNIQUE OPÉRATIONNEL (FIN MARS)

- Architecture de base implémentée
- Environnement DevOps configuré
- Pipeline CI fonctionnel
- Application déployable (structure prête)

Le socle technique est sécurisé.

J3 – AUTHENTIFICATION ET GESTION DES CYCLES FONCTIONNELLES (FIN AVRIL)

- Création de compte
- Connexion
- Gestion des cycles

Première brique métier opérationnelle.

J4 – RÉSERVATION POSSIBLE (FIN MAI)

- Gestion des prestations
- Zones configurées
- Créneaux définis
- Affectations opérationnelles

Un client peut réserver une intervention.

J5 – CYCLE COMPLET D'INTERVENTION (FIN JUIN)

- Création intervention
- Modification / Annulation
- Déclaration paiement
- Clôture

Le cycle métier principal est proche d'être complet.
Le MVP est fonctionnel.

J6 – MVP STABILISÉ (FIN JUILLET)

- Tests globaux réalisés
- Corrections appliquées
- Fonctionnalités secondaires arbitrées

Application prête pour livraison.

J7 – LIVRAISON FINALE (AOÛT)

- Démonstration
- Remise des livrables
- Soutenance

4. BACKLOG / RELEASE / SPRINT

4.1 Présentation du backlog

Le backlog produit regroupe l'ensemble des fonctionnalités identifiées à partir :

- du cahier des charges,
- des cas d'usage métier,
- du story mapping réalisé en phase de cadrage.

Il est structuré en **épiques fonctionnelles**, elles-mêmes déclinées en **user stories** orientées valeur utilisateur.

Gestion des informations de l'entreprise

- Modifier les informations de l'entreprise
- Consulter les données de l'entreprise

Cette épique concerne la gestion des données statiques de l'entreprise visibles par les utilisateurs.

Gestion des utilisateurs

- S'authentifier
- Créer un compte utilisateur
- Gérer son profil
- Administrer les utilisateurs
- Supprimer son compte

Cette épique constitue le socle d'accès sécurisé à l'application et la gestion des rôles (client, technicien, administrateur).

Gestion des cycles client

- Administrer les types de cycles
- Gérer ses cycles

Elle permet d'associer un client à un ou plusieurs cycles (VAE, Route, VTT...) afin de contextualiser les interventions.

Gestion des prestations

- Administrer les forfaits d'intervention
- Choisir un forfait d'intervention

Cette épique définit l'offre commerciale de l'entreprise et permet au client de sélectionner un service lors d'une réservation.

Gestion des produits additionnels

- Administrer les produits additionnels
- Ajouter/supprimer des produits additionnels

Elle complète l'offre principale avec des options ou produits complémentaires.

Gestion des zones géographiques

- Administrer les zones géographiques
- Gérer l'affectation des zones aux techniciens

Cette épique permet de structurer l'activité par secteur d'intervention.

Gestion de planification des créneaux

- Administrer des modèles de planification
- Définir des créneaux disponibles
- Affecter des créneaux aux zones et techniciens

Elle constitue le moteur de disponibilité du service et permet la réservation.

Gestion des interventions

- Créer une intervention
- Consulter les interventions
- Modifier une intervention avant réalisation
- Annuler une intervention
- Gérer le suivi d'une intervention
- Clôturer et gérer le paiement

Cette épique couvre le cycle métier principal : réservation → réalisation → clôture.

Gestion des exigences légales - conformité RGPD

- Consulter les documents légaux
- Accepter les CGU lors de la création de compte
- Gérer le consentement aux cookies
- Demander la suppression de son compte

Elle garantit la conformité réglementaire de l'application.

4.2 Définition de la Release MVP et priorisation

La priorisation des fonctionnalités a été réalisée à partir :

- de la valeur métier apportée,
- des dépendances techniques entre modules,
- de la nécessité de sécuriser le socle technique en amont,
- de la logique du cycle métier principal.

L'objectif est de livrer un MVP permettant un cycle complet d'intervention :
Réservation → Réalisation → Paiement → Clôture.

Le MVP inclut :

- Gestion des utilisateurs
- Gestion des cycles client
- Gestion des prestations
- Gestion des zones géographiques
- Gestion de la planification
- Gestion des interventions
- Exigences légales minimales

Deux épiques ont été volontairement exclues du périmètre initial :

- Gestion des informations de l'entreprise
- Gestion des produits additionnels

Ces fonctionnalités seront intégrées en Version 2 afin de sécuriser la livraison de la Release 1.

4.3 Organisation des sprints

L'organisation des sprints a été définie afin d'atteindre la Release MVP dans le respect de la capacité de développement estimée. Le développement est structuré en sprints mensuels permettant une progression incrémentale et maîtrisée.

Les user stories sont estimées en story points selon l'équivalence suivante :

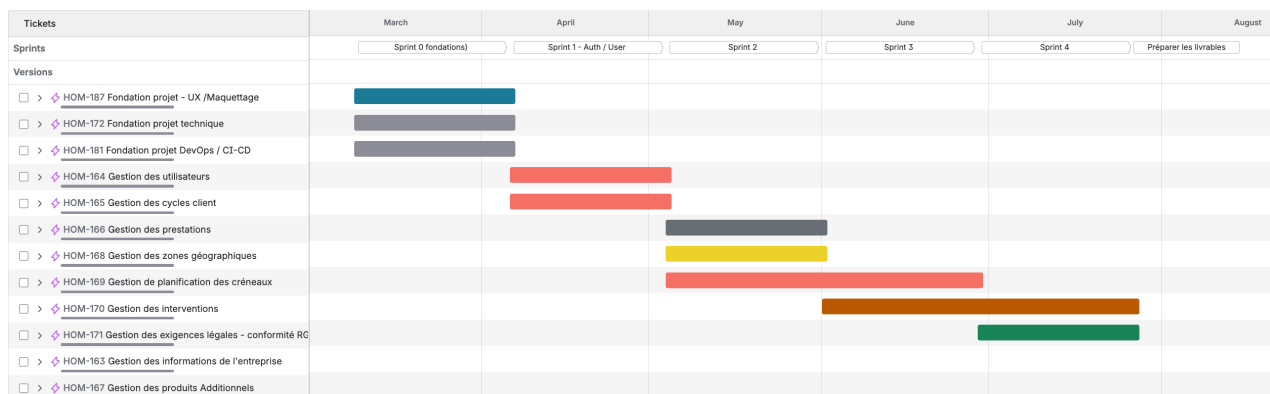
1 story point $\approx$ 1,4 heure de travail.

La capacité de développement a été estimée à 50 heures par sprint, correspondant à environ 35 story points par itération.

Cette estimation a permis de déterminer le volume maximal de fonctionnalités intégrables à chaque sprint.

Chaque sprint comprend :

- Planification
- Développement des user stories sélectionnées
- Tests fonctionnels
- Revue de sprint



Planification incrémentale des épiques et définition du périmètre MVP. Grand format en Annexe A

DÉROULEMENT DES SPRINTS

Phase 0 – Fondations (Mars)

Mise en place du socle technique et DevOps.

Sprint 1 – Authentification & cycles (Avril)

Sécurisation des accès et structuration des données.

Sprint 2 – Offre & organisation (Mai)

Implémentation des prestations et zones.

Sprint 3 – Planification & intervention (Juin)

Mise en place du moteur de réservation.

Sprint 4 – Finalisation du cycle (Juillet)

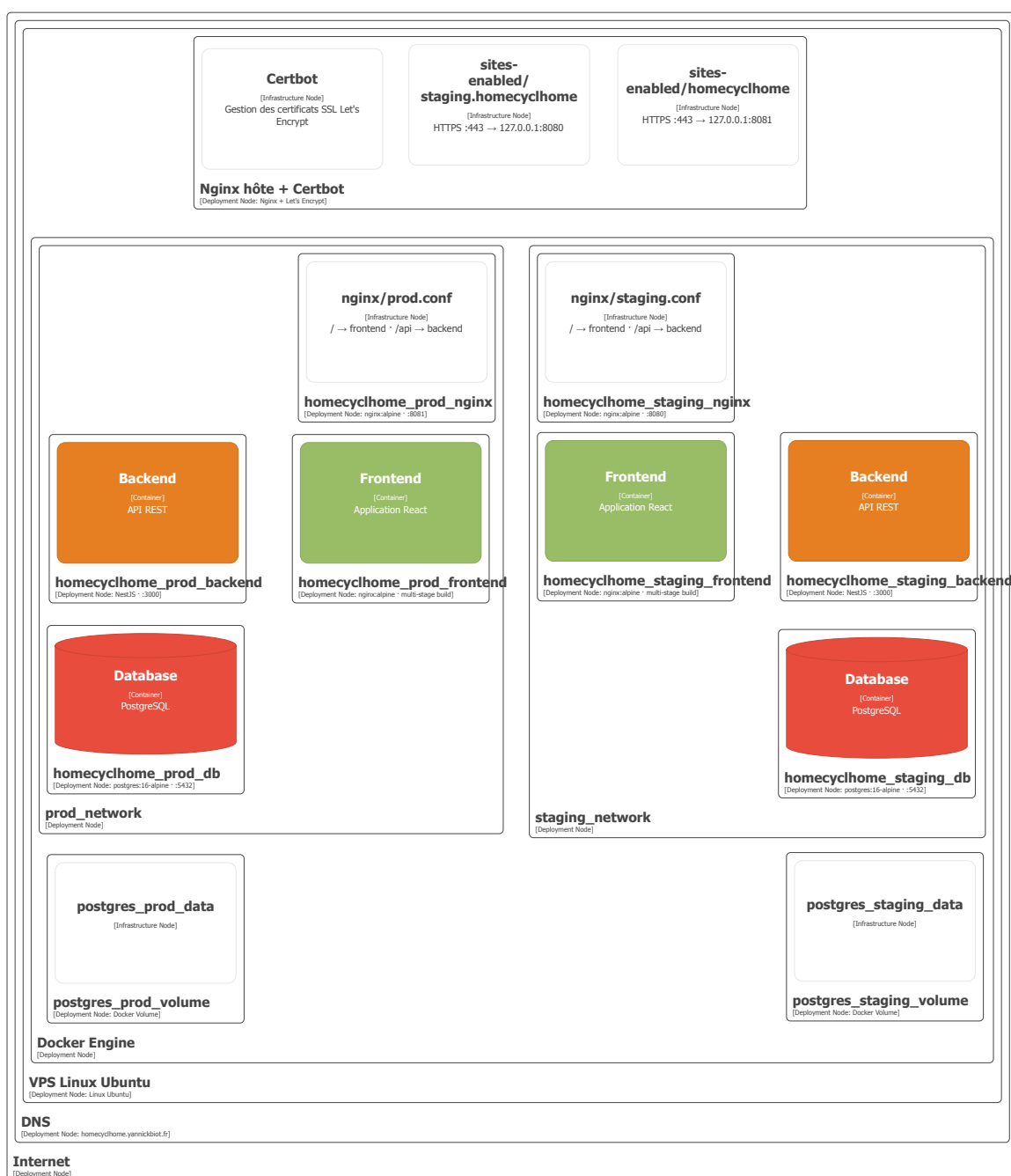
Suivi, paiement et clôture.

5. INFRASTRUCTURE

5.1 Présentation de l'infrastructure

L'application HomeCycl'Home est déployée sur un serveur cloud (VPS) exécutant Linux Ubuntu. L'infrastructure repose sur deux environnements Docker isolés — production et qualification — coexistant sur le même serveur, chacun orchestré par un fichier Docker Compose indépendant. Le point d'entrée unique de l'infrastructure est un Nginx installé directement sur le VPS. Géré par Certbot, il assure la terminaison SSL et force la redirection HTTP vers HTTPS. Il route ensuite les requêtes vers l'environnement approprié selon le sous-domaine, via les fichiers de configuration présents dans `/etc/nginx/sites-enabled/`.

5.2 Diagramme de déploiement



5.3 Description des composants

DNS : Résolution des sous-domaines `homecyclhome.yannickbiot.fr` (production) et `staging.homecyclhome.yannickbiot.fr` (qualification).

VPS : Hébergement de l'infrastructure sur Linux Ubuntu.

Nginx hôte + Certbot : Installé directement sur le VPS, il constitue le gateway principal. Il gère les certificats SSL via Let's Encrypt, force la redirection HTTP → HTTPS, et route les requêtes vers le port local de chaque stack Docker (127.0.0.1:8081 pour la production, 127.0.0.1:8080 pour la qualification). Sa configuration repose sur deux fichiers dans `/etc/nginx/sites-enabled/` : `homecyclhome` et `staging.homecyclhome`.

Nginx reverse proxy (conteneur Docker) : Présent dans chaque stack, il constitue le seul point d'entrée exposé du réseau Docker interne. Il route les requêtes `/api` vers le backend et `/` vers le frontend, en s'appuyant sur la résolution DNS interne Docker pour cibler les conteneurs par leur nom de service. Sa séparation du conteneur frontend permet de redéployer chaque service indépendamment sans interrompre la disponibilité des autres.

Frontend (conteneur Docker) : Produit par un build multi-stage — Node compile le projet React, puis une image Nginx alpine sert uniquement les fichiers statiques compilés. L'image finale ne contient ni Node ni les dépendances de développement, ce qui minimise sa taille et sa surface d'attaque.

Backend (conteneur Docker) : API REST NestJS, accessible uniquement en interne via le réseau Docker sur le port 3000.

PostgreSQL (conteneur Docker) : Base de données de chaque environnement, accessible uniquement en interne via le réseau Docker sur le port 5432.

Docker Volume : Mécanisme de persistance des données de PostgreSQL. Contrairement aux données stockées dans un conteneur — perdues à chaque redémarrage — le volume survit aux cycles de vie des conteneurs. Deux volumes distincts garantissent l'isolation des données entre environnements : `postgres_prod_data` pour la production, `postgres_staging_data` pour la qualification.

5.4 Flux réseau

Les requêtes arrivent en HTTPS sur le Nginx hôte (port 443). Les requêtes HTTP sont automatiquement redirigées en HTTPS via un code 301. Le Nginx hôte transmet ensuite la requête en HTTP vers le port local de la stack cible. Le Nginx reverse proxy interne prend le relais : il délègue `/api` au backend NestJS et `/` au conteneur frontend. Le backend communique avec PostgreSQL via le réseau interne Docker. Aucun port de base de données ou de backend n'est exposé en dehors du réseau Docker.

5.5 Sécurité et isolation

Seul le Nginx hôte est exposé publiquement sur les ports 80 et 443. Le port 80 reçoit les requêtes HTTP et les redirige immédiatement en HTTPS via un code 301 — aucun contenu n'est servi en clair. Le port 443 assure la terminaison SSL. Les conteneurs de chaque stack communiquent uniquement via leur réseau Docker bridge interne. L'isolation entre production et qualification est garantie par des réseaux Docker distincts — un conteneur de la stack production ne peut pas accéder aux services de la stack qualification, et inversement. Les volumes de données sont également séparés.

5.6 Pourquoi ces choix

La coexistence des deux environnements sur le même VPS optimise les coûts d'infrastructure tout en maintenant une isolation complète. La séparation en conteneurs distincts garantit qu'un redéploiement d'un service n'interrompt pas les autres — le frontend peut être mis à jour sans couper le reverse proxy ni l'API. Docker Compose permet d'orchestrer l'ensemble de chaque stack à partir d'un fichier de configuration unique, ce qui simplifie le déploiement et garantit la reproductibilité de l'environnement.

6. MAQUETTAGE

7. MODÉLISATION

7.1 Objectif de la modélisation

La modélisation du projet HomeCycl'Home a pour objectif de représenter de manière structurée le fonctionnement de l'application, les données manipulées ainsi que les principales interactions entre les acteurs et le système. Elle sert de base à la conception technique et au développement de la solution.

7.2 Modélisation Fonctionnelle

La modélisation fonctionnelle permet d'identifier les interactions entre les acteurs du système et l'application.

Elle est représentée par un diagramme de cas d'utilisation (UML).

Dans l'application **HomeCycl'Home**, trois acteurs principaux interagissent avec le système :

- Client
- Technicien
- Administrateur

Chaque acteur dispose de fonctionnalités spécifiques décrites dans le cahier des charges.

ACTEURS ET PRINCIPALES INTERACTIONS

Client

- Créer un compte / se connecter
- Réserver un créneau d'intervention
- Consulter ses interventions
- Annuler une intervention
- Gérer ses cycles
- Modifier son profil

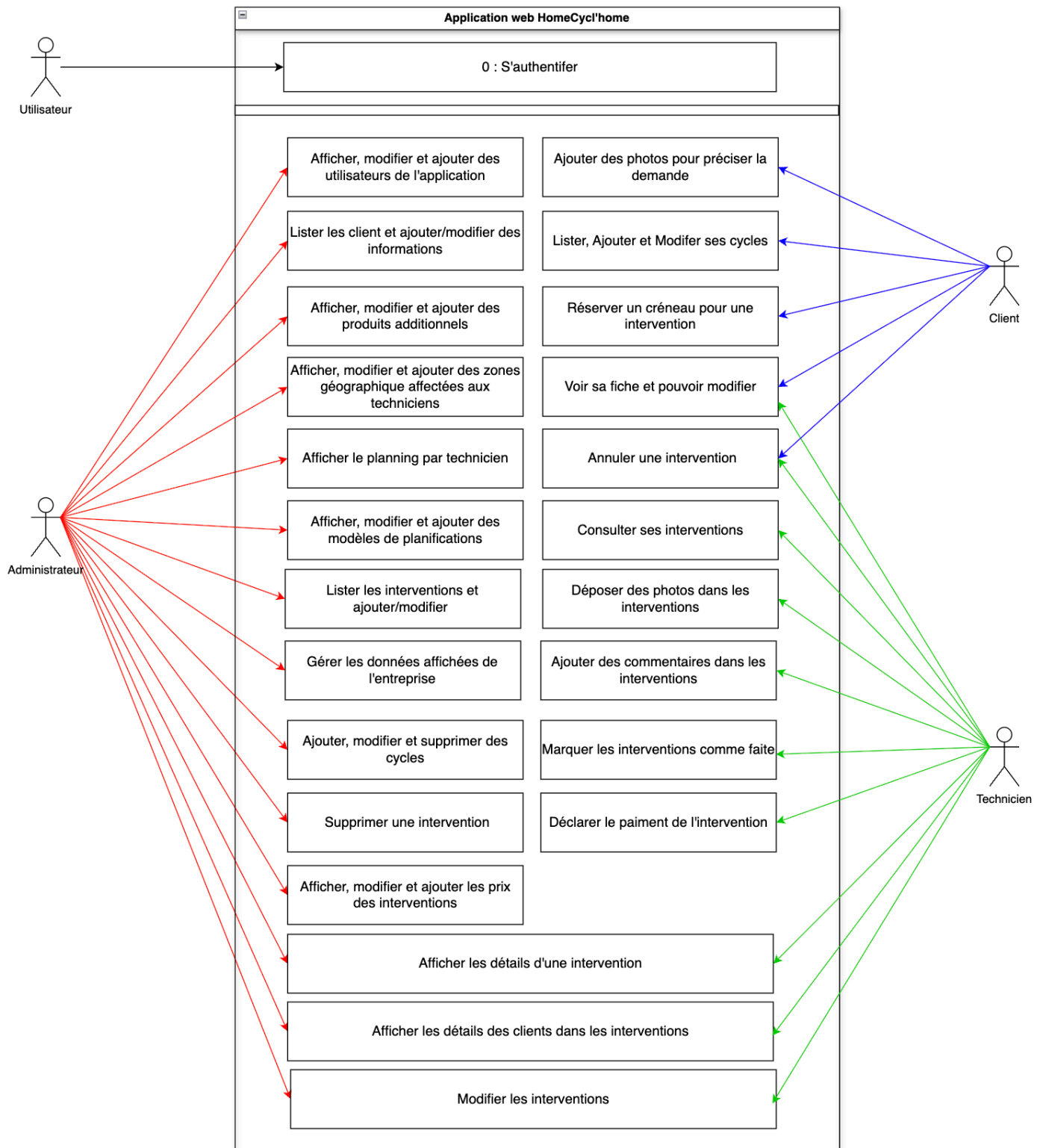
Technicien

- Consulter ses interventions
- Consulter les détails d'une intervention
- Ajouter des photos
- Ajouter des commentaires
- Marquer une intervention comme réalisée
- Annuler une intervention

Administrateur

- Gérer les utilisateurs
- Gérer les interventions
- Gérer les produits additionnels
- Gérer les forfaits d'intervention
- Gérer les zones géographiques
- Consulter le planning des techniciens

DIAGRAMME DE CAS D'UTILISATION



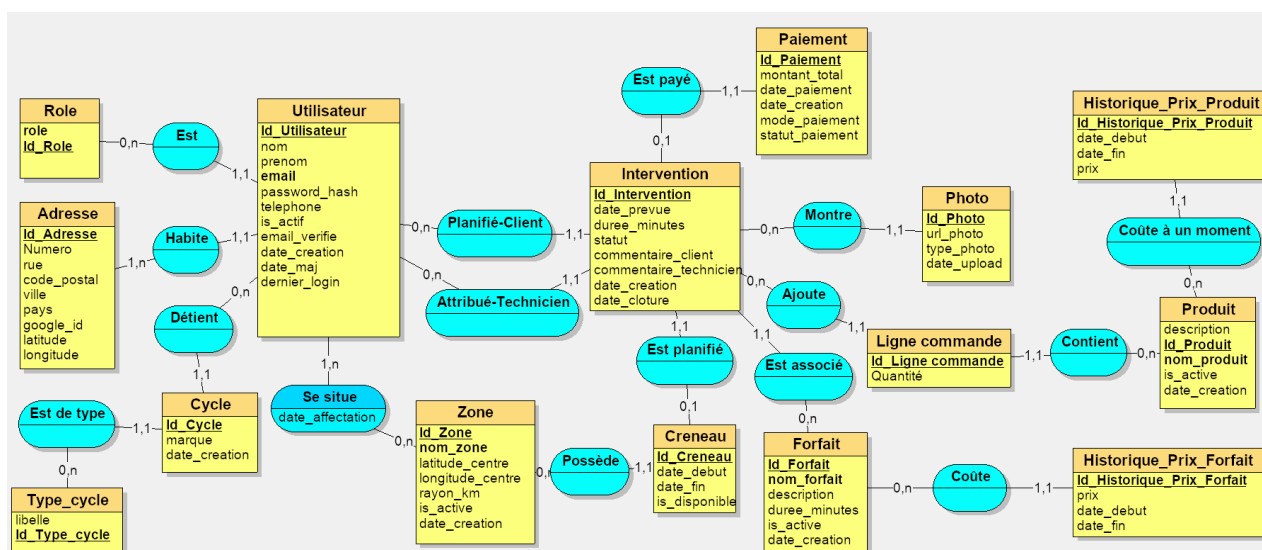
7.3 Modélisation des données

LE DICTIONNAIRE DE DONNÉES

Afin de préciser la structure des données manipulées par l'application, un dictionnaire de données a été réalisé. Celui-ci décrit l'ensemble des entités, attributs, types de données et contraintes associées au modèle. Pour des raisons de lisibilité, le dictionnaire de données complet est présenté en **annexe B** du document.

LE MODÈLE CONCEPTUEL DE DONNÉE (MCD)

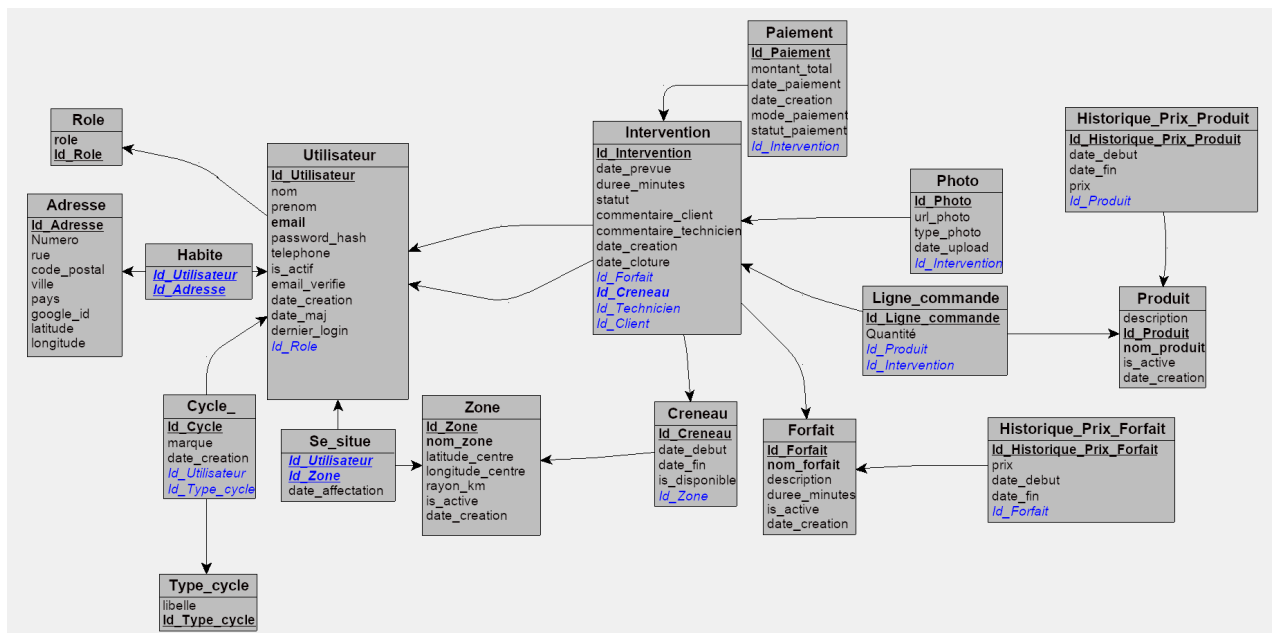
La modélisation des données vise à structurer les informations manipulées par l'application HomeCycl'Home. Elle permet d'identifier les principales entités du système, leurs attributs ainsi que les relations qui les lient. Cette modélisation constitue la base de la conception de la base de données relationnelle PostgreSQL utilisée par l'application.



En raison de la taille du modèle, une version détaillée et lisible est fournie en **annexe C**

TRANSFORMATION EN MODÈLE LOGIQUE DE DONNÉES (MLD)

Le modèle conceptuel de données a ensuite été transformé en modèle logique de données afin de préparer l'implémentation dans une base relationnelle.



En raison de la taille du modèle, une version détaillée et lisible est fournie en **annexe D**

7.4 Diagramme de séquence – Parcours client

Afin de décrire le fonctionnement dynamique du système, un diagramme de séquence a été réalisé. Celui-ci illustre les interactions entre les différents composants de l'application lors du parcours de réservation d'une intervention par un client.

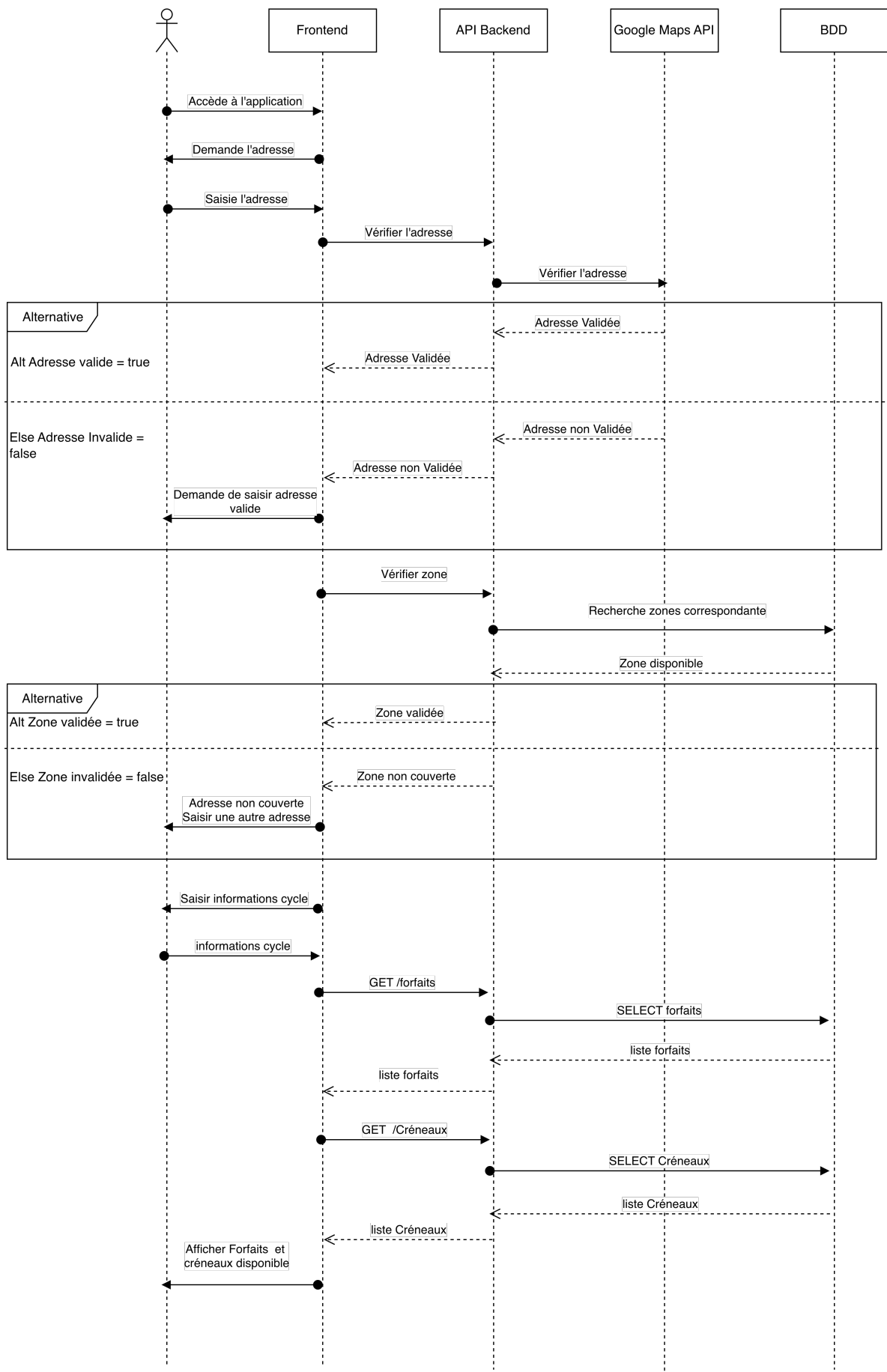
Ce diagramme met en évidence les échanges entre les éléments suivants :

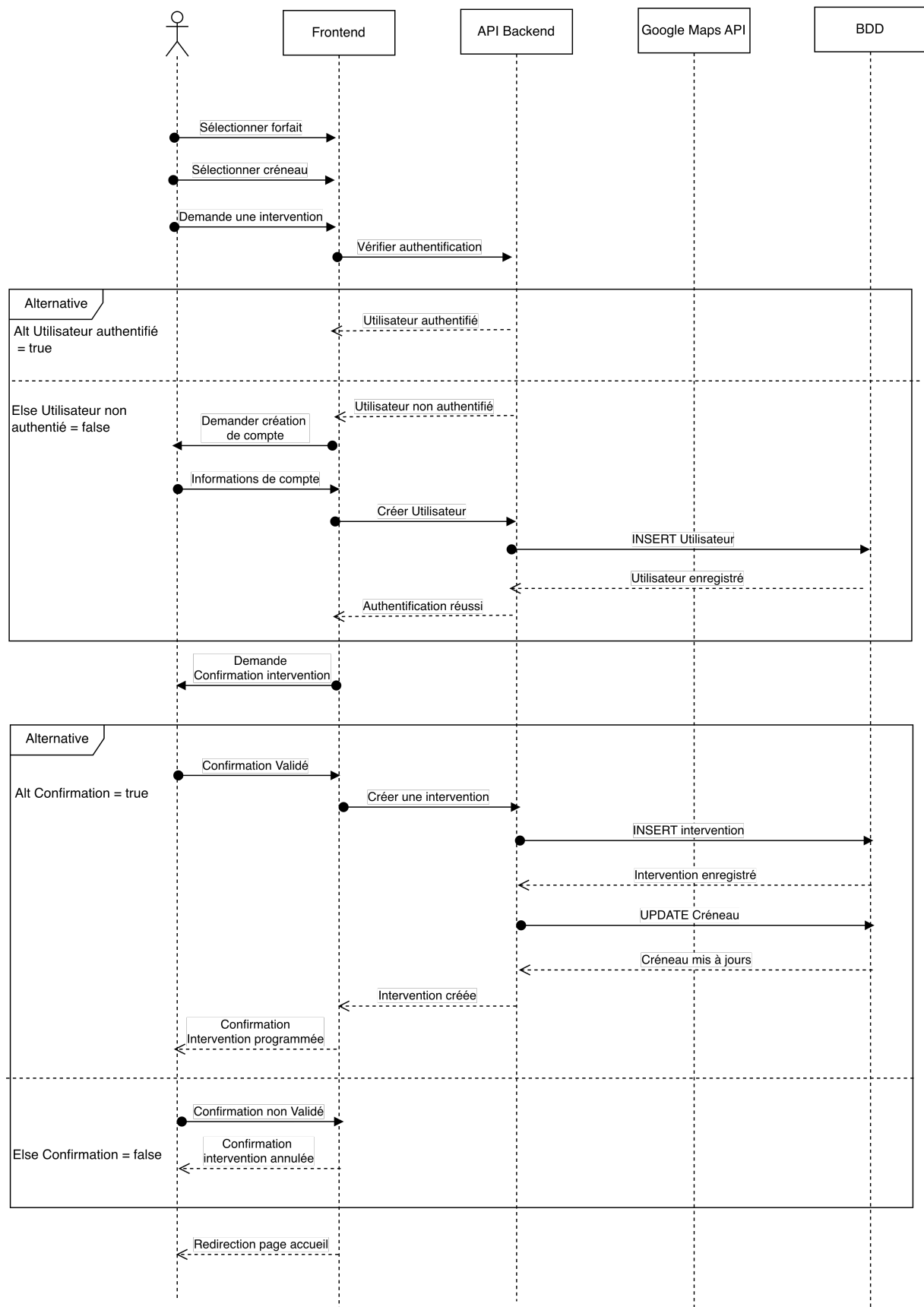
- le **client**, utilisateur de l'application,
- l'**interface frontend**, responsable de l'interaction avec l'utilisateur,
- l'**API backend**, qui gère la logique métier,
- la **base de données**, utilisée pour la persistance des informations.

Le scénario représenté correspond au parcours principal suivant :

1. le client accède à l'application et saisit son adresse ;
2. l'adresse est validée par un service externe de géolocalisation ;
3. l'application vérifie si la zone est couverte par le service ;
4. le client renseigne les informations concernant son cycle ;
5. il sélectionne un forfait puis un créneau disponible ;
6. il demande la création d'une intervention ;
7. si l'utilisateur n'est pas authentifié, il est invité à créer un compte ;
8. après confirmation du rendez-vous, l'intervention est enregistrée en base de données et le créneau est marqué comme indisponible.

Le diagramme suivant illustre les échanges nécessaires à la réalisation de ce processus.





9. APPLICATION

10. DÉPLOIEMENT EN MODE DEVOPS

10.1 Gestion du code source

HÉBERGEMENT DU CODE

Le code source du projet HomeCycl'Home est hébergé sur la plateforme GitHub. Ce choix permet de centraliser le code, de suivre l'historique des modifications et de faciliter la mise en place de processus d'intégration et de déploiement continus (CI/CD). Le dépôt constitue la source unique de vérité du projet et permet d'automatiser l'ensemble du cycle de livraison à l'aide de GitHub Actions.

STRATÉGIE DE BRANCHES

Une stratégie de branches inspirée de GitFlow simplifié est utilisée afin d'organiser le développement. Trois types de branches sont définis :

- **main** : contient la version stable de l'application déployée en production.
- **dev** : branche d'intégration dans laquelle sont regroupées les fonctionnalités en cours de développement. Elle correspond à l'environnement de qualification.
- **feature/*** : branches temporaires dédiées au développement d'une fonctionnalité spécifique.

Les fonctionnalités sont développées dans une branche feature, puis fusionnées dans dev. Une fois validées en qualification, les évolutions sont fusionnées dans main pour déclencher le déploiement en production.

WORKFLOW DE DÉVELOPPEMENT

Le cycle de développement suit le workflow suivant :

1. Création d'une branche **feature** à partir de la branche **dev**.
2. Développement de la fonctionnalité et validation locale.
3. Push de la branche feature dans github
4. Pull Request de feature -> dev
5. Vérification de l'intégration des fonctionnalités.
6. Fusion de la branche **feature** dans la branche **dev**.
7. Pull Request de dev -> main
8. Vérification de l'intégration des fonctionnalités.
9. Fusion de la branche **dev** dans la branche **main** pour préparer une nouvelle version stable.

Ce processus permet d'isoler les développements, de limiter les conflits et de maintenir une branche principale toujours stable.

CONVENTION DE COMMITS

Le projet adopte une convention de nommage des commits inspirée des "Conventional Commits". Cette convention permet de structurer l'historique des modifications et de faciliter la lecture des évolutions du projet.

Les principaux types de commits utilisés sont :

- **feat** : ajout d'une nouvelle fonctionnalité
- **fix** : correction d'un bug
- **refactor** : amélioration du code sans modification fonctionnelle
- **docs** : modification de la documentation
- **test** : ajout ou modification de tests

Exemple de message de commit :

feat: ajout de la réservation d'intervention

10.2 Intégration Continue (CI)

PRINCIPE

Le projet met en place un processus d'intégration continue via GitHub Actions. Un fichier unique ci-cd.yml orchestre l'ensemble des étapes de vérification et de déploiement. À chaque push sur les branches dev, main ou feature/*, les vérifications de qualité sont exécutées automatiquement avant tout déploiement.

CONTRÔLE DE LA QUALITÉ DU CODE

Un contrôle automatique de la qualité est effectué à chaque push via deux jobs parallèles :

- **lint-frontend** : exécute ESLint sur le code React pour détecter les erreurs et vérifier le respect des bonnes pratiques TypeScript.
- **lint-backend** : exécute ESLint sur le code NestJS.

Prettier est également utilisé pour garantir l'uniformité du formatage. Ces vérifications sont exécutées avant toute étape de build ou de déploiement — un échec bloque la pipeline.

VALIDATION DES MESSAGES DE COMMIT

La conformité des messages de commit est vérifiée localement via **Commitlint** et **Husky**. Le hook **commit-msg** s'exécute à chaque commit et bloque tout message non conforme à la convention Conventional Commits.

TESTS AUTOMATISÉS

La stratégie de tests retenue repose sur une approche TDD avec Jest pour les tests unitaires et Supertest pour les tests d'intégration API. Un job test-backend est intégré dans la pipeline et s'exécute en parallèle des jobs de lint. Les tests portent en priorité sur les règles métier critiques : gestion des créneaux, affectation des zones et contrôle des rôles. Un échec bloque la pipeline et empêche tout déploiement.

10.3 Mise en place du déploiement continu (CD)

PRINCIPE

Le déploiement est entièrement automatisé via GitHub Actions. La chaîne de livraison repose sur GitHub Container Registry (ghcr.io) pour stocker et distribuer les images Docker. Cette approche garantit que le serveur de production ne contient ni code source ni outils de build — il se limite à exécuter des images prêtes à l'emploi.

REGISTRY DOCKER

Les images Docker sont construites sur les machines GitHub Actions et publiées sur GitHub Container Registry (ghcr.io/yannou64). Deux images sont produites pour chaque environnement :

- ghcr.io/yannou64/homecyclhome-backend:prod / :staging
- ghcr.io/yannou64/homecyclhome-frontend:prod / :staging

L'authentification au registry est gérée automatiquement via le GITHUB_TOKEN pour le push depuis la pipeline, et via un Personal Access Token (PAT) stocké en secret GitHub pour le pull depuis le VPS.

PIPELINE DE DÉPLOIEMENT

Le fichier **ci-cd.yml** orchestre deux chemins de déploiement indépendants selon la branche :

- Push sur dev → build-and-push-staging → deploy-staging
- Push sur main → build-and-push-prod → deploy-prod

Chaque chemin exécute les étapes suivantes :

1. Vérification de la qualité du code (lint + tests)
2. Construction des images Docker Frontend et Backend sur GitHub Actions
3. Push des images vers ghcr.io
4. Copie des fichiers de configuration (docker-compose.yml, nginx.conf) vers le VPS via SCP
5. Connexion SSH au VPS
6. Authentification au registry (docker login ghcr.io)
7. Téléchargement des nouvelles images (docker compose pull)
8. Redémarrage des services (docker compose up -d)
9. Redémarrage du reverse proxy Nginx interne

DÉPLOIEMENT SUR LE SERVEUR

Le VPS ne contient que le strict nécessaire : les fichiers .env (secrets) et les fichiers de configuration déposés automatiquement par la pipeline à chaque déploiement. Les images sont téléchargées depuis ghcr.io et orchestrées via Docker Compose. Cette architecture garantit que le serveur de production est un environnement d'exécution pur, sans code source ni dépendances de développement.

10.4 Gestion des environnements

ENVIRONNEMENT DE DÉVELOPPEMENT

Le développement est réalisé en local sur la machine du développeur, sans utilisation de conteneurs Docker. Cette approche simplifie le cycle de développement et accélère les phases de test et de débogage.

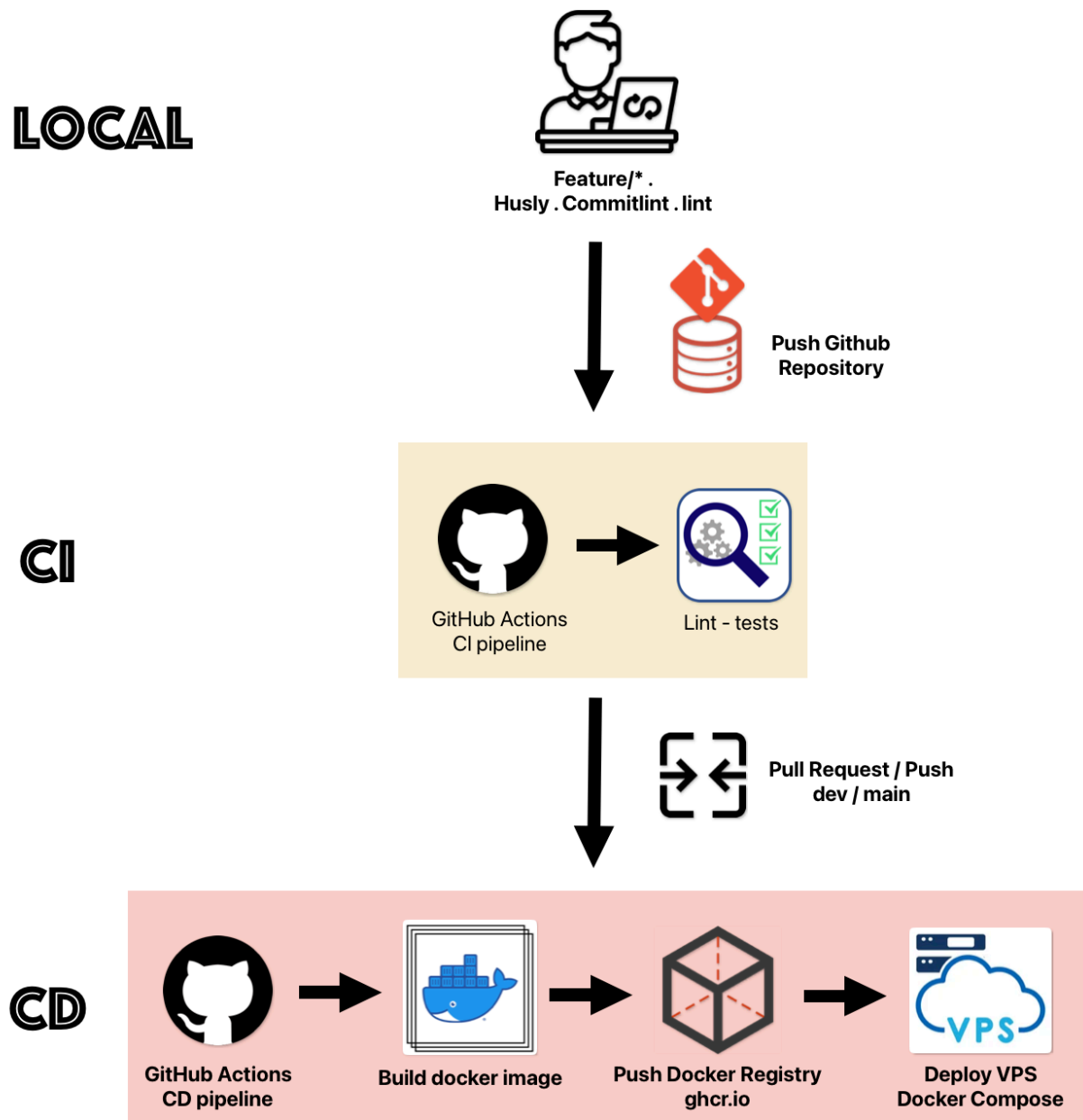
ENVIRONNEMENT DE QUALIFICATION

L'environnement de qualification est déployé sur le même VPS que la production, dans un réseau Docker isolé (staging_network). Il est accessible à l'adresse `staging.homecyclhome.yannickbiot.fr` et se déclenche automatiquement à chaque push sur la branche dev. Il permet de valider chaque évolution avant promotion en production.

ENVIRONNEMENT DE PRODUCTION

L'environnement de production est déployé sur le VPS et accessible à l'adresse `homecyclhome.yannickbiot.fr`. Il se déclenche automatiquement à chaque push sur la branche main. Les services sont orchestrés via Docker Compose et le trafic est géré par le Nginx hôte avec certificat SSL Let's Encrypt géré par Certbot.

10.5 Diagramme de déploiement



10. TESTS (UNITAIRES, FONCTIONNELS)

11. DOCUMENTATION

ANNEXE A

Tickets	March	April	May	June	July	Au
Sprints	Sprint 0 fondations)	Sprint 1 - Auth / User	Sprint 2	Sprint 3	Sprint 4	Préparer les livrables
Versions						
☐ > HOM-187 Fondation projet - UX / Maquettage						
☐ > HOM-172 Fondation projet technique						
☐ > HOM-181 Fondation projet DevOps / CI-CD						
☐ > HOM-164 Gestion des utilisateurs						
☐ > HOM-165 Gestion des cycles client						
☐ > HOM-166 Gestion des prestations						
☐ > HOM-168 Gestion des zones géographiques						
☐ > HOM-169 Gestion de planification des créneaux						
☐ > HOM-170 Gestion des interventions						
☐ > HOM-171 Gestion des exigences légales - conformité RC						
☐ > HOM-163 Gestion des informations de l'entreprise						
☐ > HOM-167 Gestion des produits Additionnels						

ANNEXE B : DICTIONNAIRE DE DONNÉES

Entité	Nom	Description	Type	Obligatoire	Règle
Utilisateur	Id_utilisateur	Identifiant technique	UUID / INT	Oui	PK, unique, non modifiable
Utilisateur	Nom	Nom de l'utilisateur	VARCHAR	Oui	
Utilisateur	Prénom	Prénom de l'utilisateur	VARCHAR	Oui	
Utilisateur	Email	Email de connexion	VARCHAR	Oui	Unique, format email
Utilisateur	Password_hash	Empreinte du mot de passe	VARCHAR	Oui	Jamais visible
Utilisateur	Telephone	Numéro de téléphone	VARCHAR	Oui	Format Regex
Utilisateur	Is_actif	Compte actif / désactivé	BOOLEAN	Oui	Défaut : true
Utilisateur	Email_verifie	Email vérifié	BOOLEAN	Oui	Défaut : false
Utilisateur	Date_creation	Création du compte	DATETIME	Oui	Auto(now)
Utilisateur	Date_maj	Dernière mise à jour	DATETIME	Oui	Auto(now)
Utilisateur	Dernier_login	Dernière connexion	DATETIME	NON	Maj à l'auth
Utilisateur	Id_role	Role de l'utilisateur	FK	Oui	Référence Rôle
Role	Id_role	Identifiant technique	UUID / INT	Oui	PK, unique, non modifiable
Role	Role	Libelle du rôle	VARCHAR	Oui	Unique
Adresse	Id_adresse	Identifiant technique	UUID / INT	Oui	PK, unique
Adresse	Numero	Numéro de voie	VARCHAR	Non	
Adresse	Rue	Rue	VARCHAR	Non	
Adresse	Code_postal	Code postal	VARCHAR	Oui	
Adresse	Ville	Ville	VARCHAR	Oui	
Adresse	google_place_id	Identifiant google	VARCHAR	Non	Unique possible
Adresse	Pays	Pays	VARCHAR	Oui	Défaut France

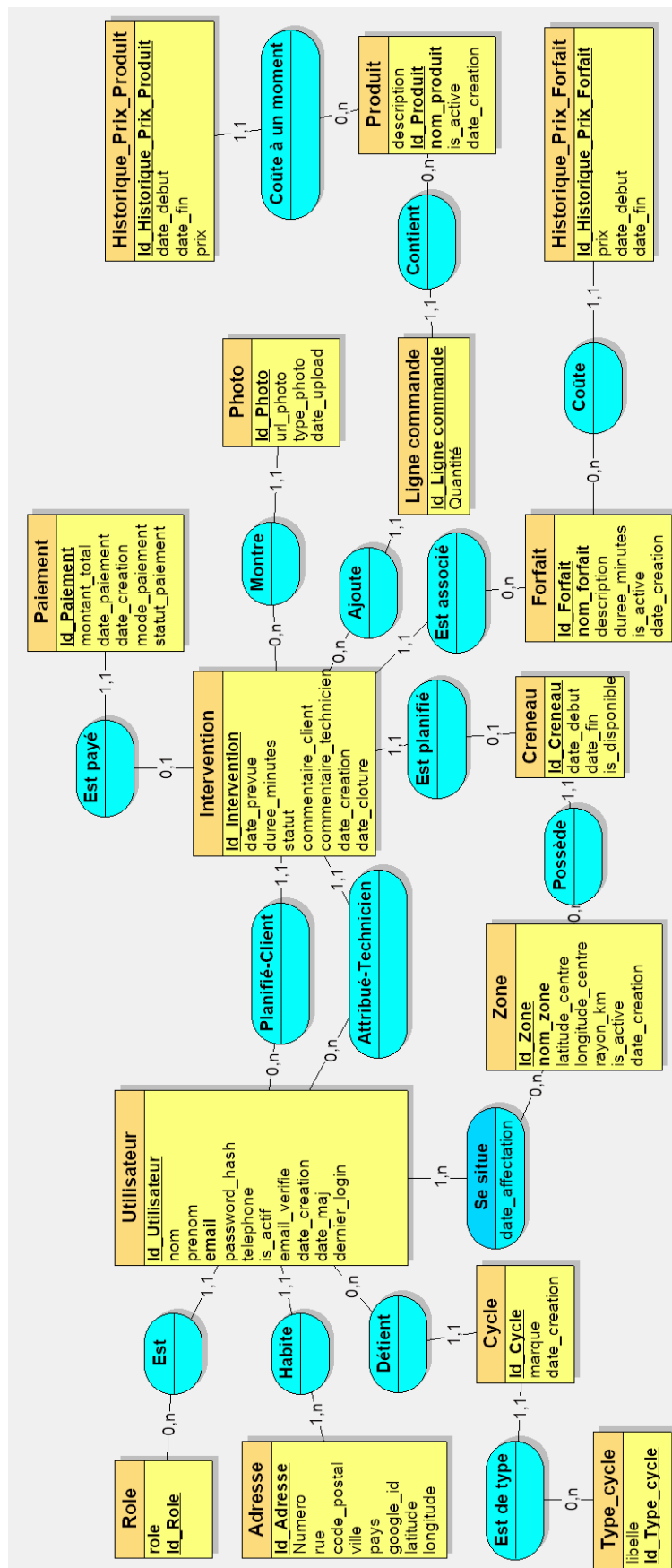
Entité	Nom	Description	Type	Obligatoire	Règle
Adresse	Latitude	Latitude	DECIMAL	Oui	Obligatoire
Adresse	Longitude	Longitude	DECIMAL	Oui	Obligatoire
Adresse	id_client	Propriétaire de l'adresse	FK	Oui	Référence Utilisateur
Zone	Id_zone	Identifiant technique	UUID / INT	Oui	PK, unique
Zone	Nom_zone	Nom de la zone	VARCHAR	Oui	Unique
Zone	Latitude_centre	Latitude centre zone	DECIMAL	Oui	
Zone	Longitude_centre	Longitude centre zone	DECIMAL	Oui	
Zone	Rayon_km	Rayon d'intervention	Int	Oui	> 0
Zone	is_active	Zone active	BOOLEAN	Oui	Défaut : true
Zone	Date_creation	Date création	DATETIME	Oui	Auto(now)
Utilisateur_Zone	Id_utilisateur	Technicien concerné	FK	Oui	Réf Utilisateur
Utilisateur_Zone	Id_zone	Zone couverte	FK	Oui	Réf Zone
Utilisateur_Zone	date_affectation	Date d'affectation	DATETIME	Oui	Auto(now)
Cycle	Id_cycle	Identifiant technique	UUID / INT	Oui	PK
Cycle	id_client	Propriétaire	FK	Oui	Réf Utilisateur
Cycle	Id_type_cycle	Type de vélo	FK	Oui	Réf Type_Cycle
Cycle	Marque	Marque	VARCHAR	NON	
Cycle	Date_creation	Date ajout	DATETIME	Oui	Auto(now)
Type_cycle	Id_type_cycle	Identifiant	UUID / INT	Oui	PK
Type_cycle	Libelle	Libelle	VARCHAR	Oui	Unique
Intervention	Id_intervention	Identifiant technique	UUID / INT	Oui	PK
Intervention	id_client	Client concerné	FK	Oui	Réf Utilisateur
Intervention	Id_cycle	Vélo concerné	FK	Oui	Réf Cycle
Intervention	Id_adresse	Adresse d'interventin	FK	Oui	Réf Adresse

Entité	Nom	Description	Type	Obligatoire	Règle
Intervention	Id_prix_forfait	Forfait sélectionné	FK	Oui	Rèf Historique_Prix_Forfait
Intervention	Id_Technicien	Technicien attribué	FK	Oui	Rèf Utilisateur (orle-technicien)
Intervention	Date_prevue	Date prévue	DATETIME	Oui	>= date du jour
Intervention	duree_minutes	Durée prévue	INT	Oui	Issue du forfait
Intervention	Statut	Statut intervention	ENUM	Oui	Planifiée, Annulée, Terminée
Intervention	Commentaire_client	Commentaire du client	TEXT	Non	
Intervention	Commentaire_technicien	Compte rendu technicien	TEXT	Non	
Intervention	Date_creation	Date création	DATETIME	Oui	Auto(now)
Intervention	Date_cloture	Date de clôture	DATETIME	Non	Si statut = Terminée
Photo	Id_photo	Identifiant	UUID / INT	Oui	PK
Photo	Id_intervention	Intervention liée	FK	Oui	Réf intervention
Photo	Url_photo	Chemin ou url	VARCHAR	Oui	
Photo	Type_photo	Type photo	ENUM	Oui	Avant, Après
Photo	Date_upload	Date upload	DATETIME	Oui	Auto(now)
Creneau	Id_creneau	Identifiant	UUID / INT	Oui	PK
Creneau	Id_zone	Zone concernée	FK	Oui	Rèf Zone
Creneau	Date_debut	Date et heure début	DATETIME	Oui	
Creneau	Date_fin	Date et heure fin	DATETIME	Oui	> date_debut
Creneau	is_disponible	Disponible	BOOLEAN	Oui	Défaut true
Forfait	Id_forfait	Identifiant technique	UUID / INT	Oui	PK
Forfait	Nom_forfait	Nom commercial	VARCHAR	Oui	Unique
Forfait	Description	Description du service	TEXT	Oui	
Forfait	Durée_minutes	Durée estimée	INT	Oui	> 0

Entité	Nom	Description	Type	Obligatoire	Règle
Forfait	is_active	Forfait actif	BOOLEAN	Oui	Défaut true
Forfait	Date_creation	Date création	DATETIME	Oui	Auto(now)
Historique_Prix_Forfait	Id_prix_forfait	Identifiant technique du prix	UUID / INT	Oui	PK
Historique_Prix_Forfait	id_forfait	Forfait concerné	FK	Oui	Rèf Forfait
Historique_Prix_Forfait	Prix	Prix TTC applicable	DECIMAL	Oui	>= 0
Historique_Prix_Forfait	Date_debut	Début de validité du prix	DATETIME	Oui	
Historique_Prix_Forfait	Date_fin	Fin de validité du prix	DATETIME	Non	Null si prix actif
Produit	Id_produit	Identifiant technique	UUID / INT	Oui	PK
Produit	Nom_produit	Nom commercial	VARCHAR	Oui	Unique
Produit	Description	Description produit	TEXT	Non	
Produit	Is_active	Produit actif	BOOLEAN	Oui	Défaut : true
Produit	Date_creation	Date création	DATETIME	Oui	Auto(now)
Ligne_commande	Id_intervention	Intervention concernée	FK	Oui	Réf Intervention
Ligne_commande	Id_prix_produit	Produit ajouté	FK	Oui	Réf Produit
Ligne_commande	Quantité	Quantité ajoutée	INT	Oui	>= 1
Historique_Prix_Produit	Id_prix_produit	Identifiant technique du prix	UUID / INT	Oui	PK
Historique_Prix_Produit	Id_produit	Produit concerné	FK	Oui	Réf Produit
Historique_Prix_Produit	Prix	Prix unitaire TTC	DECIMAL	Oui	>= 0
Historique_Prix_Produit	Date_debut	Début validité	DATETIME	Oui	
Historique_Prix_Produit	Date_fin	Fin validité	DATETIME	Non	Null si actif

Entité	Nom	Description	Type	Obligatoire	Règle
Paielement	Id_paiement	Identifiant technique	UUID / INT	Oui	PK
Paielement	Id_intervention	Intervention concernée	FK	Oui	Unique + Réf intervention
Paielement	Montant_total	Montant payé	DECIMAL	Oui	>= 0
Paielement	Mode_paiement	Mode de paiement	ENUM	Oui	Espèce, Carte, Chèque
Paielement	Statut_paiement	Statut du paiement	ENUM	Oui	En_Attente, Payé
Paielement	Date_paiement	Date du paiement	DATETIME	Non	Si payé
Paielement	Date_creation	Date enregistrement	DATETIME	Oui	Auto(now)

ANNEXE C : DIAGRAMME MCD



ANNEXE D : DIAGRAMME MLD

